

Smart Attendance and Classroom Access Platform with Face Verification and Geofenced Check-In

PID 12

Final Report

Mentor: Mr. Pasan Sarathchandra

Group Members

230702K - WICKRAMAARACHCHI W.A.A.U.

230714A - WIJAYARATHNE T.W.O.A.

230734J - WITHANA W.D.P.S.

230736R - WITHANAGE M.H.

Abstract / Executive Summary

University attendance systems based on paper registers, spreadsheets, or simple digital check-ins can be time-consuming to manage and vulnerable to proxy attendance. This project developed UniAttend, a web and Android-based attendance management platform designed to provide a secure, centralized, and auditable attendance process. The system combines authenticated identity, geofenced location validation, liveness-assisted face verification, and QR verification to provide multiple forms of attendance evidence. The completed solution includes a Next.js lecturer and administrator web dashboard, an Expo React Native student mobile application, a FastAPI core backend, a separate FastAPI face-verification service using InsightFace through ONNX Runtime, and Keycloak for authentication. Application data are stored in PostgreSQL, with Redis supporting temporary application state. Biometric embeddings are stored in encrypted form, and the platform is deployed as containerized services behind HTTPS. UniAttend supports attendance-session management, student check-in, live monitoring, attendance finalization, manual corrections, reporting, and centralized administration. Functional, interface, access-control, performance, and load testing verified the implemented workflows and security controls, with the main automated test suites completing without reported failures and normal system operations meeting the defined performance targets. The completed platform demonstrates that a multi-factor attendance workflow can be implemented using ordinary smartphones and standard server infrastructure, providing a practical approach to reducing proxy attendance while improving traceability, efficiency, and administrative control.

Table of Contents

1. Introduction.....	4
1.1 Background of the Application Domain / Problem.....	4
1.2 Motivation for the Selected System Development.....	4
1.3 Importance and Main Purpose of the System.....	4
1.4 System Overview, Approach and Outcome.....	4
2. Literature Review.....	4
3. System Models.....	6
3.1 System Requirements.....	6
3.2 System Design.....	7
3.3 Database Design.....	10
4. System Implementation.....	11
4.1 Implementation Procedure.....	11
4.2 Materials.....	11
4.3 The Algorithm.....	12
4.4 Main Interfaces.....	13
5. System Testing and Analysis.....	14
5.1 Testing Approach.....	14
5.2 Unit Testing, Results and Analysis of Testing.....	15
5.3 Performance, Security and Failure-Related Analysis.....	16
5.4 System Usability Study.....	17
6. Conclusion and Future Work.....	18
References.....	18

1. Introduction

1.1 Background of the Application Domain / Problem

Attendance records support academic monitoring and reporting, yet many institutions still rely on paper registers, spreadsheets or basic digital forms. These methods consume lecture time, are difficult to consolidate and may allow proxy attendance. A reliable digital platform must therefore verify the student, confirm that attendance is recorded at the intended class and time, and maintain traceable records for review and reporting.

1.2 Motivation for the Selected System Development

Single-factor attendance methods have limitations: GPS can be less precise indoors, QR codes can be shared, and face verification alone cannot confirm classroom presence. UniAttend was therefore designed around complementary verification methods. The initial check-in combines authenticated identity, geofence validation, liveness evidence, and face verification, while lecturers can activate short-lived QR verification during the lecture.

1.3 Importance and Main Purpose of the System

The purpose of UniAttend is to provide centralized and auditable attendance management without requiring dedicated biometric hardware or continuous camera monitoring. Students receive a guided mobile attendance workflow and attendance history, lecturers manage sessions and monitor attendance, and administrators manage academic data, users, system policies, and audit information.

1.4 System Overview, Approach and Outcome

The completed UniAttend platform follows a client-server architecture consisting of a student Android application, a lecturer and administrator web dashboard, a Core API for attendance and academic operations, and a separate Face Verification service. Keycloak provides centralized authentication and role-based access, while PostgreSQL stores persistent application data and Redis manages temporary application state. The server-side components are deployed as containerized services on a VPS, with Caddy acting as the HTTPS ingress for external requests. The platform integrates attendance-session management, geofence and face verification, QR verification, attendance finalization, manual corrections, reporting, and auditing within a single system.

2. Literature Review

Digital attendance systems aim to reduce the administrative effort associated with paper registers and spreadsheets while improving the availability and consistency of attendance records. However, the reliability of an attendance record depends on how accurately the system establishes both student identity and physical presence. Individual verification methods have limitations: GPS accuracy can decrease indoors, QR codes may be shared, and face verification alone cannot confirm

that a student is inside the required classroom. Combining complementary verification methods can therefore provide stronger attendance evidence than relying on a single mechanism.

Commercial attendance systems apply different combinations of these technologies. Jibble and Truein mainly target employee and multi-site workforce attendance and provide features such as facial recognition, GPS tracking, and geofencing [1], [2]. Qwickly Attendance and SEAtS ONE are more closely related to educational environments and support attendance methods including QR scanning, location confirmation, Wi-Fi, Bluetooth, and card-based check-in [3], [4]. These systems show that digital attendance solutions commonly combine identity or location-related controls according to their operating context.

Research has also explored face-based classroom attendance. AttenFace uses classroom camera images and face recognition to identify students for attendance [5]. In a related study, Babu et al. combine geofencing and facial recognition by validating the student's GPS location before comparing a facial embedding with a stored reference [6]. Their work demonstrates the benefit of combining identity and location verification, although liveness detection was identified as future work in that study.

The reviewed systems show that no single verification technique is sufficient for every attendance scenario. Location-based approaches provide evidence of physical proximity but provide limited identity assurance, while face-based approaches verify identity without independently confirming classroom presence. QR-based techniques are convenient for large groups but require controls against sharing or reuse. A multi-evidence approach can therefore provide stronger attendance evidence by combining mechanisms that address different weaknesses.

UniAttend adopts this principle for a university-specific, student-initiated attendance workflow. The initial check-in combines authenticated user identity, course and session validation, geofenced location verification, liveness-assisted face capture, and one-to-one face verification. During an active lecture, the lecturer can additionally activate short-lived QR verification, allowing classroom presence to be reconfirmed when required. The collected evidence is then used by the backend during attendance finalization rather than treating each successful verification step as an independent final attendance record. The system also provides lecturer monitoring, authorised manual corrections, reporting, and audit records as part of the same attendance-management platform.

Compared with the reviewed systems, the main distinction of UniAttend is therefore the integration of several complementary forms of attendance evidence within a single university attendance workflow. Among the reviewed systems, the cited works demonstrate individual or partial combinations of face recognition, geofencing, QR verification, and other presence mechanisms, while UniAttend combines authenticated identity, geofence validation, liveness-assisted face verification, lecturer-controlled QR verification, and backend attendance finalization. This design aims to reduce the weaknesses associated with relying on a single verification mechanism while maintaining a practical check-in process that can operate using ordinary student smartphones and standard server infrastructure.

3. System Models

3.1 System Requirements

The system has three main roles. Students use the Android application to authenticate, view sessions, complete required attendance checks, scan QR codes when requested, view attendance history, and receive attendance-related notifications. Lecturers use the web dashboard to create, activate, monitor and close sessions, initiate QR verification, review failed or uncertain attendance cases, apply authorised manual attendance and generate reports. Administrators manage accounts, academic data, classrooms, geofences, system policies, reference faces, corrections and audit information. The backend enforces role permissions, student enrolment, attendance timing, duplicate prevention, verification rules and attendance finalization.

Non-functional requirements focus on security, privacy, reliability, usability and performance. Protected requests require valid authentication and role-based authorization, while external communication is secured through HTTPS. Biometric embeddings are stored in encrypted form. Permanent application records are stored in PostgreSQL, while Redis is used only for temporary application state. Public access is restricted through the HTTPS ingress, while internal application and data-service ports remain private. The user interfaces provide clear guidance when permission, location, QR, or face-capture checks fail. The supported student mobile platform is Android.

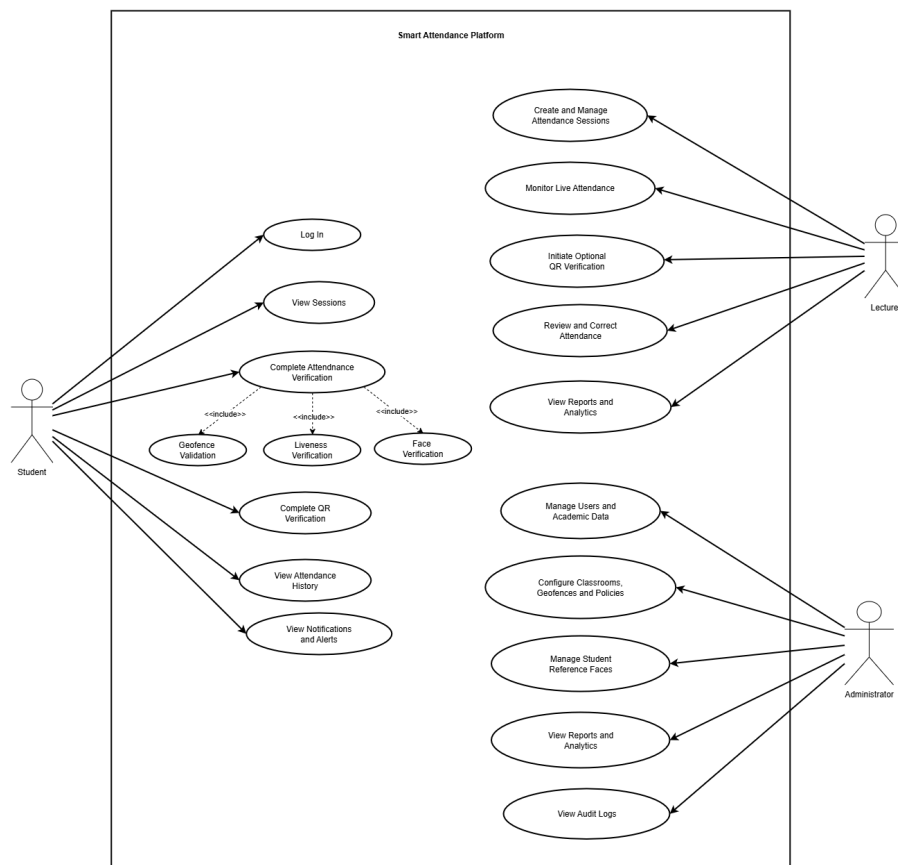


Fig. 1. Main use-case diagram of UniAttend

3.2 System Design

UniAttend follows a client-server architecture with a service-oriented modular backend. Fig. 2 presents the implemented system architecture and the main communication paths between the client applications, backend services, authentication service, and data stores.

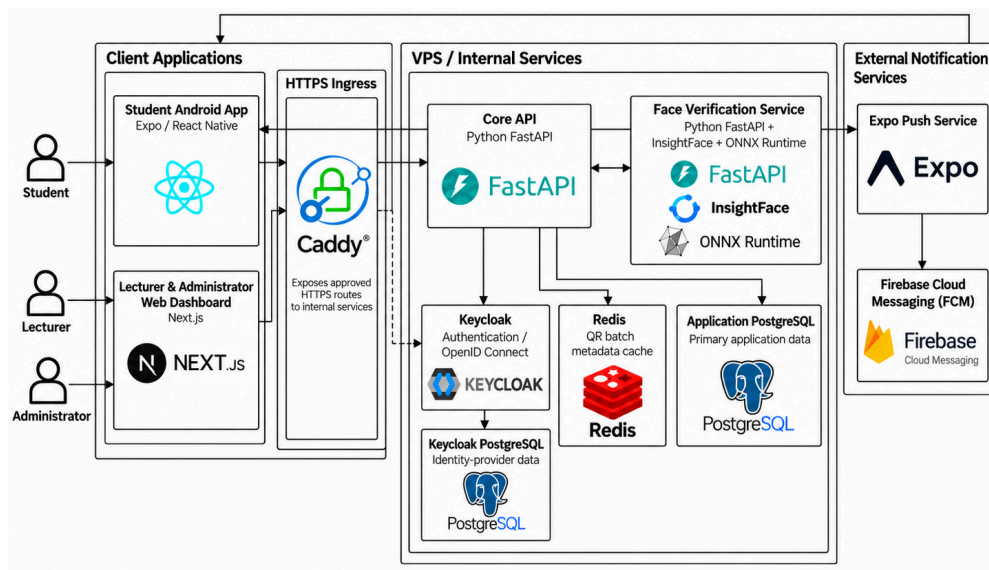


Fig. 2. Implemented system architecture

Students access the system through the Android application, while lecturers and administrators use the web dashboard. External requests enter through Caddy, which provides HTTPS ingress and forwards approved requests to the internal services. Authentication is handled by Keycloak using OpenID Connect [8], while the FastAPI Core API validates access tokens and maps the authenticated Keycloak subject to the corresponding application user. This separates identity management from attendance business logic.

The Core API coordinates academic data, attendance sessions, geofence and QR validation, attendance records, correction requests, reporting, notifications, and auditing. Face processing is isolated in a separate FastAPI Face Verification Service that uses InsightFace through ONNX Runtime [9], [10] and performs one-to-one comparison against encrypted reference embeddings. The Core API also integrates with external push-notification services to deliver attendance-related notifications to the student Android application. Permanent application data is stored in a dedicated PostgreSQL database on the VPS, while Keycloak uses a separate PostgreSQL database on the same VPS for authentication and identity-provider data. Redis is used for short-lived application states. Internal services communicate through private Docker networking, while Caddy exposes only the required HTTPS endpoints. The separate Face Verification Service also isolates the more computationally intensive biometric workload from the Core API.

Fig. 3 shows the main domain classes and their relationships.

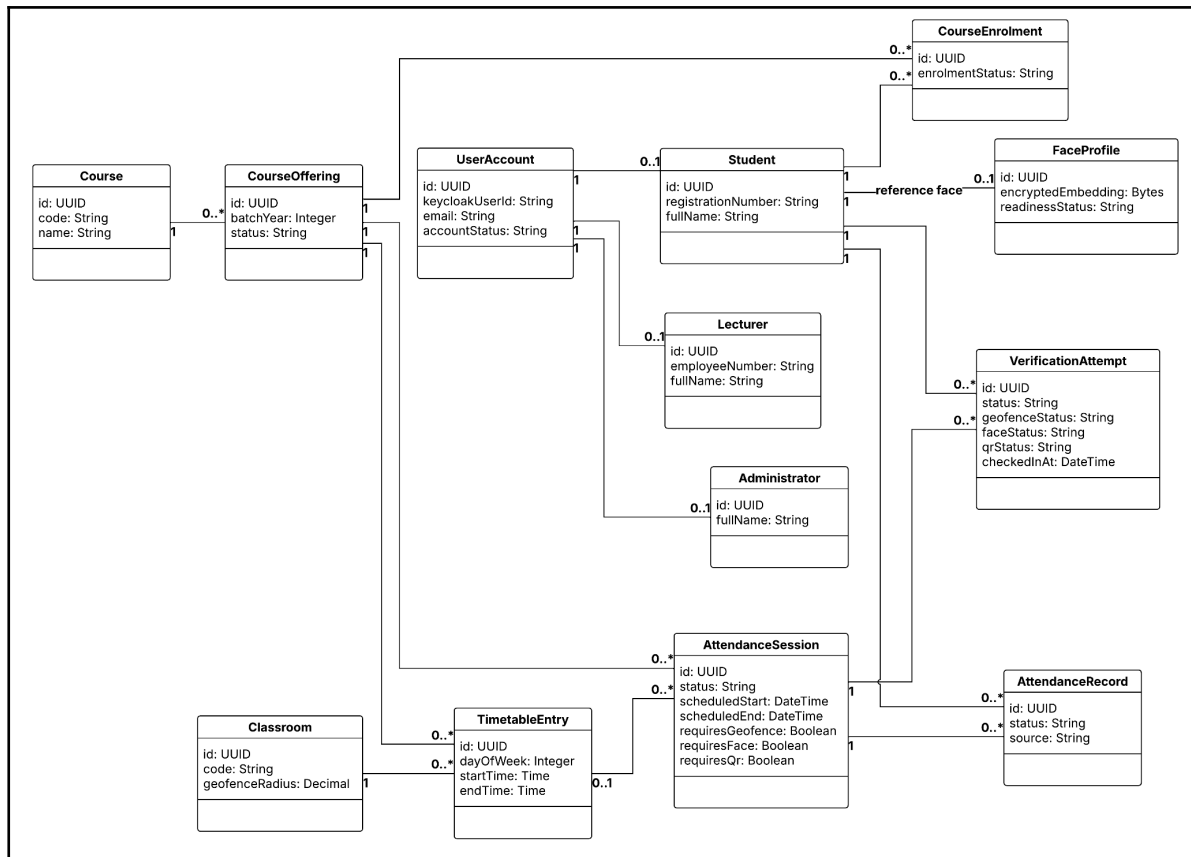


Fig. 3. Simplified core-domain class diagram

The core domain classes include users and role-specific profiles, courses and course offerings, student enrolments, timetables, classrooms, attendance sessions, verification attempts, face profiles, geofence and QR verification records, attendance records, correction requests, and audit records. AttendanceSession represents a scheduled attendance event for a lecture, verification-related entities store the results of checks such as geofence, face, and QR verification. AttendanceRecord stores the final attendance status of each student for that session.

Fig. 4 presents the main student attendance check-in and verification workflow.

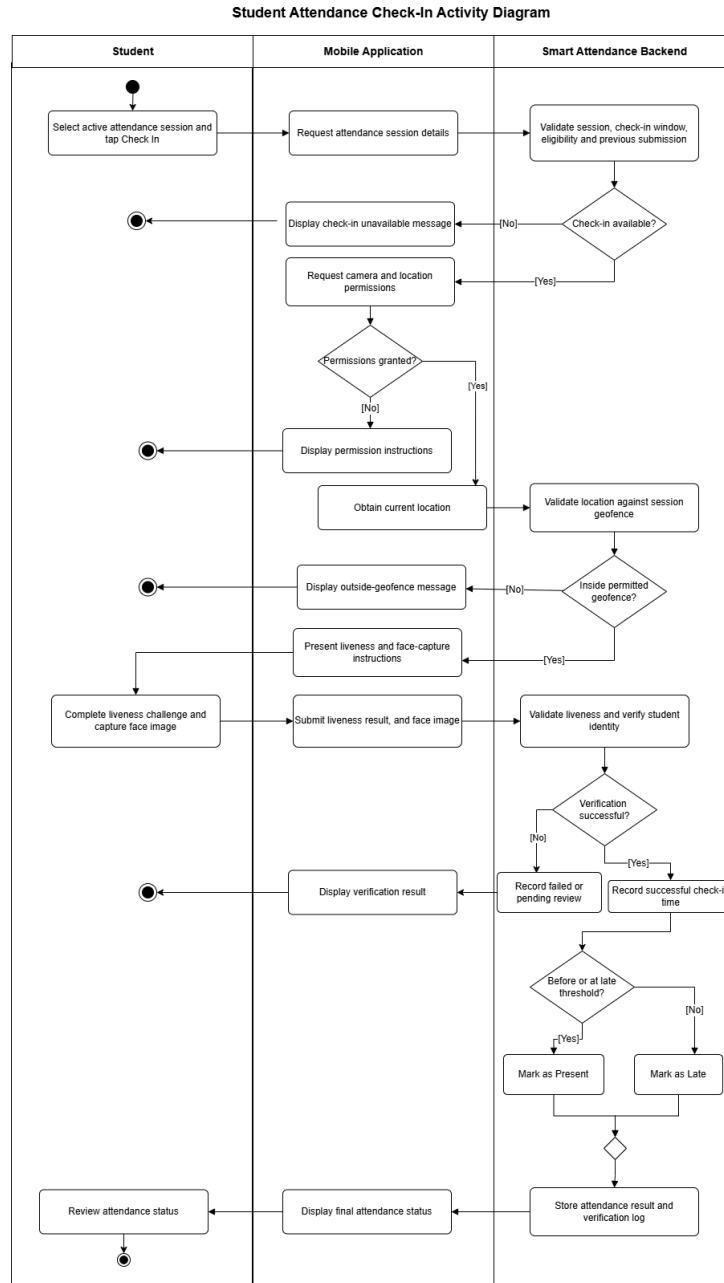


Fig. 4. Student attendance check-in and verification workflow

The student attendance workflow begins when an authenticated student selects an active attendance session. The Core API validates the student's identity, course enrollment, and attendance timing before processing the required verification steps. Geofence validation confirms location, while liveness-assisted face verification is performed through the separate Face Verification Service when required. During an active lecture, the lecturer can initiate QR verification, and the student's successful QR result is recorded as additional attendance evidence. When the session is closed or finalized, the backend evaluates the required evidence and stores the student's final attendance status.

3.3 Database Design

UniAttend separates application data from identity-provider data. Academic, attendance-session, verification-attempt, reporting, notification, and audit records are stored in the primary PostgreSQL application database deployed on the VPS. The Core API and Face Verification Service access this database through encrypted TLS connections. Keycloak maintains its authentication and identity-provider data in a separate PostgreSQL database deployed with Keycloak on the VPS.

Each UniAttend user has a permanent internal application user ID, while the corresponding Keycloak subject identifier is stored separately and used to map authenticated users to their application records. Face profiles store encrypted reference biometric embeddings together with the metadata required for verification. Redis is used as a temporary cache for QR batch metadata, allowing frequently accessed QR-session information to be retrieved without repeatedly querying PostgreSQL. The database remains the authoritative source, and the QR workflow falls back to PostgreSQL when Redis is unavailable.



Fig. 5. Updated simplified entity-relationship diagram

4. System Implementation

4.1 Implementation Procedure

Development was managed in a shared GitHub monorepo containing the web application, Android application, backend services, database scripts, deployment configuration, and automated tests. The web dashboard was implemented using Next.js and TypeScript, while the student application was developed with React Native and Expo. The Core API and Face Verification Service were implemented using Python FastAPI. PostgreSQL was used for persistent relational data, while Redis was integrated as a temporary cache for QR batch metadata. Automated checks for the main components were executed through GitHub Actions as part of the development and release workflow.

Authentication was integrated using Keycloak with OpenID Connect [8]. The Android application uses the authorization-code flow with PKCE, while protected backend requests are validated using signed access tokens before role-based authorization is applied. Face verification was implemented in a separate service using InsightFace [9] through ONNX Runtime [10]. Mobile-side face observations are used to support lightweight challenge-response liveness checks through ML Kit [7], while OpenCV provides image-processing operations where required. Reference face embeddings are encrypted before being stored.

The server-side system was packaged and deployed using Docker [11] on a Debian-based VPS. Caddy provides the HTTPS entry point, while application services remain behind controlled internal network boundaries. Releases are tied to an exact merged Git commit, validated through automated checks and service health checks before public access is enabled. The Android application is built through Expo Application Services and configured to communicate with the deployed API and Keycloak endpoints.

4.2 Materials

The system uses structured academic and attendance-related data, including student and lecturer accounts, courses, course offerings, course enrolments, timetable entries, classrooms, attendance sessions, and geofence information. Classroom locations are represented using geographic coordinates, an allowed geofence radius, and location-accuracy limits. Face-verification testing also requires authorised student reference photographs to generate the reference biometric data used for identity verification.

Development and testing were carried out using controlled mock or synthetic records rather than direct access to the university's production data. These records were arranged to represent realistic relationships between students, lecturers, courses, classrooms, and attendance sessions. At the time of final testing, the database contained 10 students, 7 lecturers, 19 courses, 17 course offerings, 12 classrooms, and 36 attendance sessions used for system validation.

Additional synthetic accounts were used during load and performance testing so that repeated and high-volume requests could be generated without using real student credentials. Test records also

covered different attendance conditions such as successful check-in, late attendance, QR verification, failed verification, and attendance corrections. Sensitive credentials, biometric data, reference photographs, and private location coordinates are not included in this report.

4.3 The Algorithm

The main attendance algorithm combines authenticated identity, session state, enrolment, geofence evidence, liveness-assisted face verification, optional QR verification, and attendance finalization. The implementation-neutral pseudocode in Fig. 6 summarizes the main decision process.

Algorithm: UniAttend Attendance Decision Workflow

STEP 1: Initial Check-In

```
Verify the authenticated user and resolve the student account.
IF the student is inactive or not enrolled in the session
    RETURN Rejected.
```

```
Verify that the attendance session is active and the check-in window is open.
IF an attendance result already exists
    RETURN the existing result.
```

```
IF geofence verification is required
    Validate the student's location.
    IF location verification fails
        RETURN Failed or Retry Required.
```

```
IF face verification is required
    Validate the liveness evidence.
    IF liveness verification fails
        RETURN Failed or Retry Required.
```

```
Compare the captured face with the stored reference embedding.
IF face verification fails
    RETURN Failed or Retry Required.
```

```
IF all required initial checks are satisfied
    IF completion time exceeds the late threshold
        Record initial status as Late Check-In.
    ELSE
        Record initial status as Checked-In.
```

STEP 2: QR Verification

```
IF the lecturer activates QR verification
    Verify that the student has completed the initial check-in.
    Validate the submitted QR token.
    Record the QR verification result.
```

STEP 3: Attendance Finalization

```
WHEN the lecturer closes or finalizes the session
    FOR EACH enrolled student
        IF an authorized manual attendance decision exists
            Preserve the manual decision.
        ELSE IF the required attendance evidence is incomplete
            Record Absent.
        ELSE IF the initial check-in was late
            Record Late.
        ELSE
            Record Present.
```

STEP 4: Manual Review

```
IF a failed or uncertain case is reviewed by an authorized lecturer
```

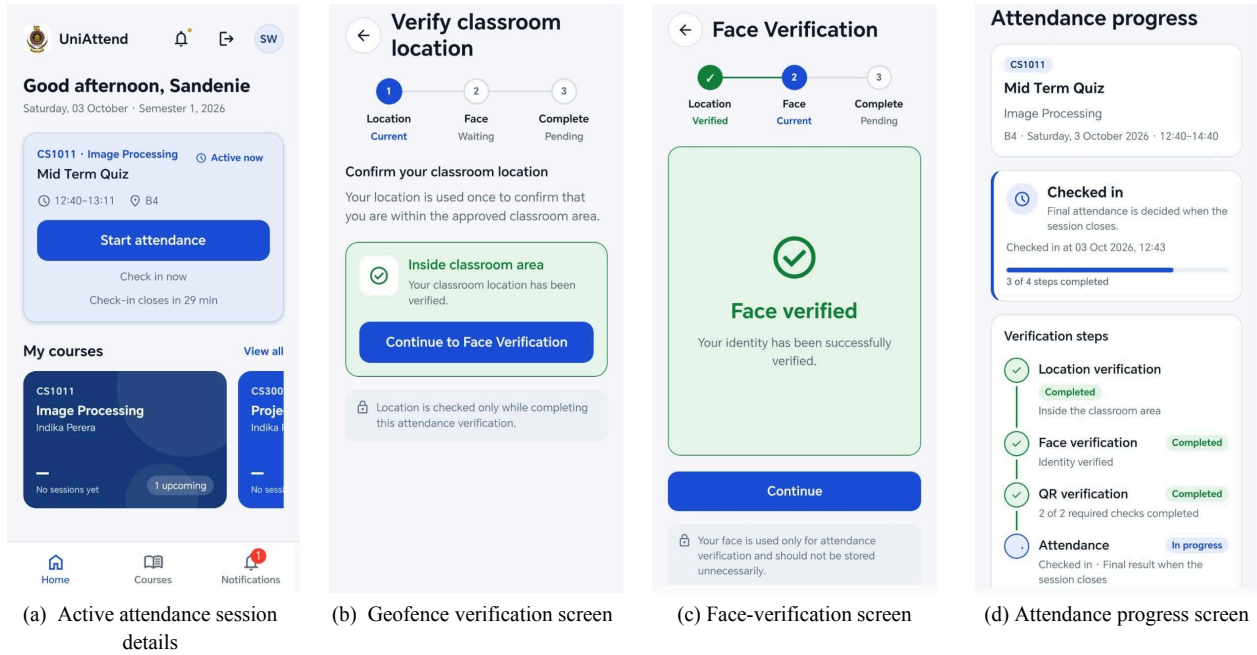



Fig. 8. Main student attendance interfaces

Fig. 8 shows the main student attendance workflow in the Android application. Students can view active sessions, complete location and face-verification checks, scan a lecturer-provided QR code when required, and view the resulting attendance status. The QR verification step is designed as a short interaction and can typically be completed within approximately 2–3 seconds, allowing students to provide additional attendance evidence with minimal disruption during an active lecture.

5. System Testing and Analysis

5.1 Testing Approach

Testing followed a layered strategy in which automated component and service tests were supported by PostgreSQL integration tests, manual API checks, interface testing, access-control testing, performance profiling, and deployment validation. Testing included both successful and failure scenarios across authentication and authorization, attendance-session management, geofence validation, QR verification, face and liveness checks, attendance finalization, manual attendance changes, correction requests, input validation, and duplicate-prevention rules.

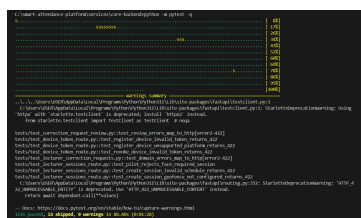
Security testing focused on access control because Student, Lecturer, and Administrator roles use the same backend services. A matrix of 11 critical endpoints was tested using both permitted and denied roles. Additional cases covered invalid or expired tokens, QR misuse, liveness failures, and protection of biometric information, guided by OWASP security concerns [12]. Web accessibility was evaluated using Lighthouse and WCAG-based criteria [14], while performance and load testing were carried out using k6 [13].

5.2 Unit Testing, Results and Analysis of Testing

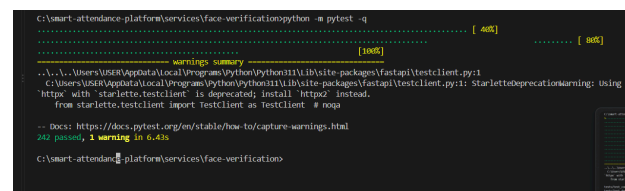
The automated test suites covered the Core API, Face Verification Service, web application, and Android application. The latest test execution completed 2,067 tests, of which 2,054 passed, 13 were skipped, and none failed. Table 1 summarizes the results.

Component	Executed	Passed	Failed	Skipped
Core API	1,129	1,116	0	13
Face Verification	242	242	0	0
Web	157	157	0	0
Android	539	539	0	0
Total	2,067	2,054	0	13

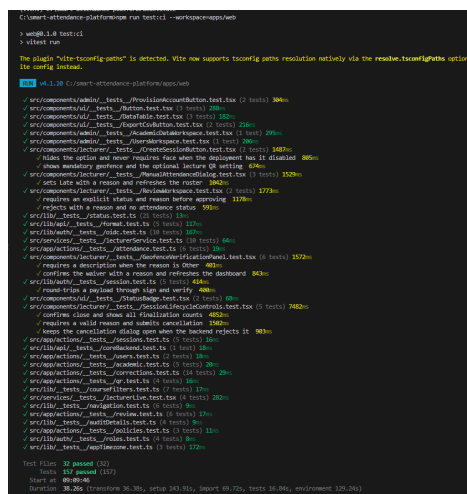
Table 1. Summary of automated test results



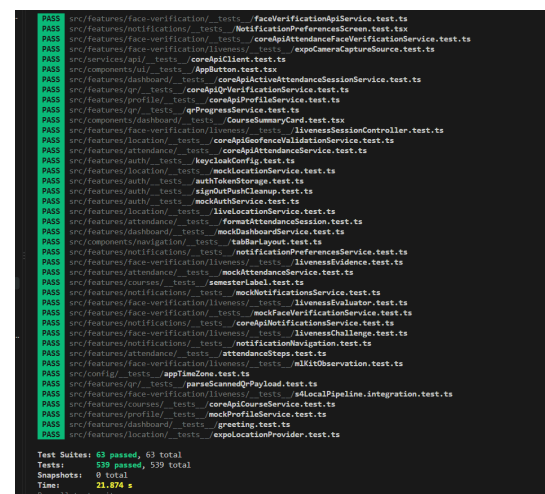
(a) Core API test results



(b) Face Verification test results



(c) Web test results



(d) Android test results

Fig. 9. Automated test evidence for the Core API, Face Verification Service, web application, and Android application.

All four automated test suites completed without failures. The 13 skipped tests occurred in the Core API suite, while the Face Verification, web, and Android suites completed with all executed tests passing. Functional testing covered authentication and authorization, attendance-session management, geofence and QR verification, face and liveness checks, attendance finalization, manual attendance changes, and correction workflows. The 11-endpoint access-control matrix also reported no authorization mismatches.

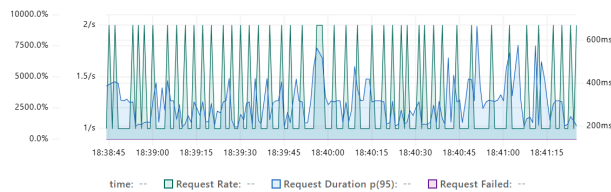
5.3 Performance, Security and Failure-Related Analysis

Performance and load testing evaluated response times under normal, simultaneous, and sustained workloads. Table 2 summarizes the main results.

Scenario	p95 result	Outcome
Mobile PKCE login	~1 s	Target met
Student profile	470 ms	Target met
Face verification	~2 s	Below 10 s target
20 simultaneous logins	2.32 s	Target met
50 simultaneous logins	4.25 s	3 s target exceeded; 0 transaction failures
100 simultaneous logins	9.55 s	3 s target exceeded; 0 transaction failures
200 simultaneous logins	21.89 s	3 s target exceeded; 0 transaction failures
Controlled 200 logins/min	2.10 s	Target met

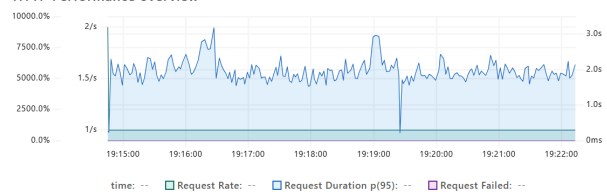
Table 2. Summary of performance and load-test results

HTTP Performance overview



(a) single-user response-time profile

HTTP Performance overview



(b) face-verification performance profile

scenario	target_logins	completed_logins	failures	min_ms	average_ms	median_ms	p95_ms	max_ms	result
Simultaneous 20	20	20	0	1357	1884.6	1946	2321.05	2322	Pass
Simultaneous 50	50	50	0	1586	3074.44	3168	4248.9	4377	Fail
Simultaneous 100	100	100	0	5434	7033.94	6466	9550	9944	Fail
Simultaneous 200	200	200	0	15569	18622.56	18557	21893.40	22561	Fail
Controlled 200/min	200	201	0	661	955.89	723	2098	2507	Pass

(c) authentication load-test results

Fig. 10. Performance and load-test evidence

The results show that normal single-user operations remained within the defined performance targets. Authentication also remained stable under the controlled workload of 200 login arrivals per minute. However, simultaneous bursts of 50 or more users exceeded the 3 s authentication response-time target, although no login transactions failed. Face verification remained within its 10 s target. The face-verification test reused a protected test image and therefore measures processing latency and consistency rather than biometric accuracy across different users or environmental conditions.

Security is supported through Keycloak authentication, role-based backend authorization, HTTPS communication, private container networking, and encrypted reference face embeddings. PostgreSQL remains the authoritative application data store, while Redis is used as a temporary cache for QR batch metadata, with database fallback when the cache is unavailable. Access-control testing of 11 critical endpoints reported no authorization mismatches. Transaction handling, validation rules, service health checks, and backup procedures further support data consistency and recovery.

5.4 System Usability Study

A System Usability Scale (SUS) study was conducted to evaluate the usability of UniAttend. Participants completed representative tasks using both the student mobile application and the lecturer web dashboard, and then answered the standard 10-item SUS questionnaire using a five-point Likert scale. The study focused on how easily users could understand and complete the main student and lecturer workflows. Participants were asked to perform tasks such as signing in and locating an attendance session, completing the student attendance workflow, and using the lecturer interface to manage an attendance session.

Measure	Result
Participants	20
Mean SUS score	78.25/100
Highest score	95.0
Lowest score	55.0
Participants completing assigned tasks	20/20

Table 3. Summary of System Usability Scale results

Participant	Statement 1	Statement 2	Statement 3	Statement 4	Statement 5	Statement 6	Statement 7	Statement 8	Statement 9	Statement 10	
Pearson	5	2	3	3	3	2	2	4	3	3	55.0
Pearson	5	2	5	2	5	1	5	1	5	1	95.0
Pearson	5	1	5	1	1	1	5	1	5	5	80.0
Pearson	5	2	4	2	3	1	5	3	4	2	77.5
MSc 1	5	1	4	2	4	1	4	4	4	2	77.5
MSc 2	5	2	4	1	4	2	4	3	5	1	82.5
MSc 3	5	1	5	5	5	5	5	5	5	5	60.0
MSc 4	4	3	4	2	4	2	4	3	4	2	70.0
MSc 5	4	2	4	3	4	2	4	2	4	4	67.5
MSc 10	5	2	4	2	4	2	5	1	4	2	82.5
MSc 11	4	1	4	3	5	1	5	1	4	3	82.5
MSc 12	5	1	5	1	5	1	5	5	5	1	90.0
MSc 13	4	2	4	2	4	2	4	2	5	1	80.0
MSc 14	5	1	4	2	5	1	5	2	5	1	92.5
MSc 15	4	1	5	1	4	1	5	1	4	1	92.5
MSc 16	5	4	4	1	3	2	3	2	4	4	65.0
MSc 17	5	2	4	2	4	2	5	1	4	2	82.5
MSc 18	4	2	4	2	4	2	4	2	4	2	75.0
MSc 19	5	2	4	2	4	2	5	1	4	3	80.0
MSc 20	5	3	4	2	5	1	3	1	4	3	77.5
AVERAGE											78.25

Fig. 11. SUS scores obtained from the usability study

The study produced a mean SUS score of 78.25/100, exceeding the target value of 68 defined for the study. Most participants completed the assigned tasks without assistance, indicating that the main UniAttend workflows were generally easy to understand and use.

6. Conclusion and Future Work

UniAttend was developed as a secure and centralized attendance management platform for university environments. The completed system combines authenticated identity, enrolment and session validation, geofence verification, liveness-assisted face verification, and lecturer-controlled QR verification to provide multiple forms of attendance evidence. It includes an Android student application, lecturer and administrator web dashboards, FastAPI backend services, Keycloak authentication, PostgreSQL data storage, Redis-supported QR caching, and containerized deployment behind HTTPS. The system also supports attendance-session management, live monitoring, attendance finalization, manual attendance changes, correction requests, reporting, and audit records.

Testing confirmed the main functional workflows, role-based access controls, client applications, and deployed services. The automated test suites completed without reported failures, while performance testing showed that normal login, profile-access, and face-verification operations remained within their defined response-time targets. Load testing also demonstrated stable sustained authentication throughput, although large simultaneous login bursts produced increased response times. Overall, the project demonstrates that face, location, QR, and authenticated identity verification can be combined into a practical attendance workflow using standard smartphones and server infrastructure.

Future work could extend UniAttend through larger-scale biometric accuracy studies using a diverse and consented dataset, stronger server-bound liveness challenges, and broader performance testing under mixed classroom workloads. Additional enhancements could include support for iOS devices, integration with university student-information or learning-management systems, expanded operational monitoring and recovery exercises, and further refinement of privacy and biometric-data retention policies. These improvements would support future adoption of UniAttend across larger academic environments. Overall, UniAttend demonstrates that multiple complementary verification methods can provide a fast and practical attendance workflow while improving confidence in the recorded attendance decision.

References

- [1] Jibble, “Free Attendance Management Software,” Jibble. [Online]. Available: <https://www.jibble.io/attendance-management>. [Accessed: Jun. 28, 2026].
- [2] Truein, “Geofencing Attendance System | GPS Based Attendance App for Field Staff,” Truein. [Online]. Available: <https://truein.com/geofencing-attendance-system>. [Accessed: Jun. 28, 2026].
- [3] Qwickly, “Qwickly Attendance Mobile App Setup,” Qwickly Help Center. [Online]. Available: <https://qwickly.zendesk.com/hc/en-us/articles/13295304911501-Qwickly-Attendance-Mobile-App-Setup>. [Accessed: Jun. 28, 2026].
- [4] SEAtS, “Student Attendance Management Made Easy Today,” SEAtS ONE. [Online]. Available: <https://seatsone.ai/student-attendance-management/>. [Accessed: Jun. 28, 2026].

- [5] A. Rao, “AttenFace: A Real Time Attendance System using Face Recognition,” arXiv preprint arXiv:2211.07582, Nov. 2022.
- [6] L. K. Babu, N. V. Prasad, S. Yeswanth and E. P. Kumar, “Smart Attendance System using Geofencing and Facial Recognition,” Int. J. Progressive Res. Eng. Manage. Sci., vol. 5, no. 11, pp. 291-297, Nov. 2025.
- [7] Google, “ML Kit Face Detection.” [Online]. Available: <https://developers.google.com/ml-kit/vision/face-detection>. [Accessed: Jul. 27, 2026].
- [8] Keycloak, “Keycloak Documentation.” [Online]. Available: <https://www.keycloak.org/documentation>. [Accessed: Jul. 27, 2026].
- [9] DeepInsight, “InsightFace: 2D and 3D Face Analysis Project,” GitHub repository. [Online]. Available: <https://github.com/deepinsight/insightface>. [Accessed: Aug. 10, 2026].
- [10] Microsoft, “ONNX Runtime Documentation.” [Online]. Available: <https://onnxruntime.ai/docs/>. [Accessed: Aug. 10, 2026].
- [11] Docker, “Docker Documentation.” [Online]. Available: <https://docs.docker.com/>. [Accessed: Aug. 15, 2026].
- [12] OWASP Foundation, “OWASP Top 10:2021.” [Online]. Available: <https://top10.owasp.org/2021/>. [Accessed: Jul. 20, 2026].
- [13] Grafana Labs, “k6 Documentation - API Load Testing and Thresholds.” [Online]. Available: <https://grafana.com/docs/k6/latest/>. [Accessed: Sep. 27, 2026].
- [14] World Wide Web Consortium, “Web Content Accessibility Guidelines (WCAG) 2.2,” W3C Recommendation, Dec. 2024. [Online]. Available: <https://www.w3.org/TR/WCAG22/>. [Accessed: Jul. 20, 2026].